

? LlamaIndex & Agentic AI - Classroom Topics

? LlamaIndex & Agentic AI - Classroom Topics

Topics covered in this notebook

1. LlamaIndex vs LangChain
2. Vector Index
3. Tree Index
4. Keyword Index
5. Knowledge Graph
6. Sub-Question Query Engine
7. Agno and Agent Memory
8. LlamaIndex vs LangChain vs Agno

The detailed theory, examples, and executable code for each topic appear in the corresponding section below.

? TOPIC 2 - Vector Index

LlamaIndex

LlamaIndex is an open-source framework that helps developers connect Large Language Models (LLMs) like GPT, Claude, Gemini, or Llama with their own data (PDFs, Word documents, databases, APIs, Notion, Slack, etc.).

It is one of the most popular frameworks for building:

- * Retrieval-Augmented Generation (RAG) applications
- * AI Agents

What Problem Does It Solve?

LLMs only know what they were trained on and do not automatically know about your private or up-to-date information.

LlamaIndex bridges that gap by:

1. Loading your data (PDFs, documents, SQL databases, APIs, etc.)
2. Creating searchable indexes (typically vector indexes)
3. Retrieving the most relevant information when a user asks a question
4. Sending that context to the LLM so it can generate an accurate answer

Example Workflow

? Classroom Theory - LlamaIndex

What is LlamaIndex?

LlamaIndex is an open-source framework used to connect an LLM with external/private data.

An LLM may know general information from its training data, but it does not automatically know the contents of our companys PDFs, policies, databases, internal documents, or other data sources.

LlamaIndex provides building blocks for:

- Loading external data
- Splitting data into manageable chunks
- Creating embeddings
- Building indexes
- Retrieving relevant information
- Sending retrieved context to an LLM
- Building query and chat interfaces
- Connecting multiple knowledge sources
- Building more advanced agent/tool workflows

The key classroom idea

LlamaIndex is not the LLM.

Think of it as the data and retrieval layer around the LLM.

RAG in one sentence

RAG = Retrieve relevant information first, then ask the LLM to generate an answer using that information.

Basic RAG flow

Trainer explanation

A useful way to explain this is:

The LLM is the brain that generates the answer. LlamaIndex helps the brain access the right information at the right time.

Demo code for LlamaIndex

? Classroom Theory - Installation and Environment Setup

Before building the application, we install the LlamaIndex packages required by the demo.

Packages used

- * ? core LlamaIndex functionality
- * ? Groq LLM integration
- * ? Hugging Face embedding integration
- * ? supporting library used by the Hugging Face embedding model

Important security point

API keys should not be hard-coded in a notebook that may be shared.

For classroom demos, use:

- Google Colab Secrets
- Environment variables
- A file when appropriate

The updated notebook uses rather than storing the actual secret in source code.

Code

```
# Install the required Python package in the current notebook environment.
!pip -q install llama-index
# Install the required Python package in the current notebook environment.
!pip -q install llama-index-llms-groq
# Install the required Python package in the current notebook environment.
!pip -q install llama-index-embeddings-huggingface
# Install the required Python package in the current notebook environment.
!pip -q install sentence-transformers
```

Output

```
???????????????????????????????????????????? 11.9/11.9 MB 25.9 MB/s eta 0:00:00
???????????????????????????????????????????? 1.8/1.8 MB 20.0 MB/s eta 0:00:00
???????????????????????????????????????????? 166.8/166.8 kB 4.7 MB/s eta 0:00:00
???????????????????????????????????????????? 51.0/51.0 kB 1.4 MB/s eta 0:00:00
???????????????????????????????????????????? 169.4/169.4 kB 4.9 MB/s eta 0:00:00
[?25hWARNING: huggingface-hub 1.29.0 does not provide the extra 'inference'
```

Code

```
# Import the os module so its functionality is available in this notebook.
import os
# Import userdata from google.colab so we can use it directly.
from google.colab import userdata
# Read/set the API-key environment variable so the LLM can authenticate without hard-coding
the secret.
os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")
```

? Classroom Theory - Creating the Knowledge Base

Before RAG can retrieve anything, we need source data.

For this first demonstration, the notebook creates a small text file containing information about LlamaIndex.

In a real enterprise application, this data could instead come from:

- PDFs
- Word documents
- Websites
- Databases
- APIs
- Notion
- Slack
- Cloud storage

Why create a local text file?

It makes the first demo simple. Students can clearly see:

Source document ? LlamaIndex ? searchable index ? answer

This is intentionally a small knowledge base so the RAG mechanics are easy to understand.

Code

```

# Existing comment/documentation line; it explains or labels the following code.
## Create some sample data.
# Import the os module so its functionality is available in this notebook.
import os
# Create the directory if it does not already exist.
os.makedirs("data", exist_ok=True)
# Store the sample knowledge-base text in a Python string.
text = """
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
LlamaIndex is an open-source framework for connecting LLMs with external data.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
It supports:
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
- Retrieval-Augmented Generation (RAG)
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
- Vector Databases
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
- Document Indexing
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
- AI Agents
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
- Query Engines
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
- Chat Engines
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
LlamaIndex works with OpenAI, Groq, Anthropic, Gemini and many other LLM providers.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"""
# Open the specified file for the requested operation; the with block closes it automatically.
with open("data/llamaindex.txt", "w") as f:
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
f.write(text)
# Display the result/status in the notebook so learners can see what happened.
print("Sample document created.")

```

Output

Sample document created.

? Classroom Theory - LlamaIndex Components

This section imports the main components used by the basic RAG pipeline.

Important components

- Loads documents from a directory.
- Converts source files into LlamaIndex objects.
- Creates a vector-based searchable index.
- It allows semantic retrieval rather than relying only on exact keyword matching.
- Provides global configuration for the LLM, embedding model, and other LlamaIndex components.

Groq

- Connects LlamaIndex to a Groq-hosted language model.
- Converts text into numerical vectors called embeddings.

Key distinction

The LLM generates language.

The embedding model represents meaning numerically.

The index makes those representations searchable.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
## Building the index
# Import ( from llama_index.core so we can use it directly.
from llama_index.core import (
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
VectorStoreIndex,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
SimpleDirectoryReader,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Settings,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
# Import Groq from llama_index.llms.groq so we can use it directly.
from llama_index.llms.groq import Groq
# Import HuggingFaceEmbedding from llama_index.embeddings.huggingface so we can use it
directly.
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
```

? Classroom Theory - Configuring the LLM

Here we configure the language model that will finally generate the response.

The important conceptual separation is:

The notebook uses Groq as the LLM provider.

The selected model is the generation engine; LlamaIndex handles the surrounding retrieval/query workflow.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# Configure Groq LLM
# Configure the global LlamaIndex setting used by subsequent indexing/query operations.
Settings.llm = Groq(
# Specify the LLM model name used for generation.
model="openai/gpt-oss-120b")
```

? Classroom Theory - Embeddings

An embedding converts text into a numerical vector.

For example, conceptually:

Two pieces of text with similar meaning tend to have vectors that are close in the embedding space.

This is why semantic search can find:

How does retrieval augmented generation work?

even when the document contains:

RAG retrieves relevant context before generation.

`BAAI/bge-small-en-v1.5`

The notebook uses this Hugging Face embedding model to create those vectors.

Classroom takeaway

Embeddings are for representing meaning; the LLM is for generating the response.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# Configure embedding model
# Configure the global LlamaIndex setting used by subsequent indexing/query operations.
Settings.embed_model = HuggingFaceEmbedding(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
model_name="BAAI/bge-small-en-v1.5"
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
```

Output

```
modules.json: 0%|          | 0.00/349 [00:00<?, ?B/s]
```

Error Output

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Output

```
config_sentence_transformers.json: 0%|          | 0.00/124 [00:00<?, ?B/s]
```

Output

```
README.md: 0%|          | 0.00/94.8k [00:00<?, ?B/s]
```

Output

```
sentence_bert_config.json:  0%|          | 0.00/52.0 [00:00<?, ?B/s]
```

Output

```
config.json:  0%|          | 0.00/743 [00:00<?, ?B/s]
```

Output

```
model.safetensors: reconstructing file:  0%|          | 0.00B / 133MB
```

Output

```
model.safetensors: downloading bytes:      | 0.00B
```

Output

```
Loading weights:  0%|          | 0/199 [00:00<?, ?it/s]
```

Output

```
tokenizer_config.json:  0%|          | 0.00/366 [00:00<?, ?B/s]
```

Output

```
vocab.txt:  0%|          | 0.00/232k [00:00<?, ?B/s]
```

Output

```
tokenizer.json: 0%|          | 0.00/711k [00:00<?, ?B/s]
```

Output

```
special_tokens_map.json: 0%|          | 0.00/125 [00:00<?, ?B/s]
```

Output

```
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]
```

? Classroom Theory - Loading Documents

looks inside the specified directory and loads the available files.

The reader converts source files into LlamaIndex document objects.

Conceptually:

This is the ingestion stage of the RAG pipeline.

Code

```
# Existing comment/documentation line; it explains or labels the following code.  
# Load documents  
# Load the files from the data directory into LlamaIndex Document objects.  
documents = SimpleDirectoryReader("data").load_data()
```

Error Output

```
WARNING:llama_index.core.readers.file.base:`llama-index-readers-file` package not found, some
```

file readers will not be available if not provided by the `file\_extractor` parameter.

Code

```
# Display the result/status in the notebook so learners can see what happened.  
print("Documents loaded:", len(documents))
```

Output

```
Documents loaded: 1
```

? Classroom Theory - Building a Vector Index

is one of the most important lines in the basic RAG demo.

Conceptually, LlamaIndex performs a pipeline similar to:

The resulting index allows us to search the source material semantically.

Important

Creating an index does not mean the LLM has memorized the documents.

The documents remain external knowledge that can be retrieved when a question is asked.

Code

```
# Build a vector index from the loaded documents; this creates the searchable representation.  
index = VectorStoreIndex.from_documents(documents)
```

Code

```
# Display the result/status in the notebook so learners can see what happened.  
print("Index built successfully!")
```

Output

```
Index built successfully!
```

? Classroom Theory - Query Engine

A query engine is the interface through which we ask questions against the index.

Typical flow:

This is the core RAG behavior demonstrated in the notebook.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
## Query the index
# Create a query engine that retrieves relevant context and uses the configured LLM to answer.
query_engine = index.as_query_engine()
# Send the question to the query engine so it can retrieve relevant context and generate an
answer.
query_engine.query("What is LlamaIndex?").response
```

Output

```
'LlamaIndex is an open?source framework that bridges large language models (LLMs) with
```

external data sources. It enables capabilities such as retrieval-augmented generation,

vector?database integration, document indexing, AI agents, query engines, and chat engines,

and it works with a variety of LLM providers including OpenAI, Groq, Anthropic, Gemini, and

others. '

Code

```

# Define several example questions to demonstrate repeated querying.
questions = [
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"What is Retrieval-Augmented Generation?",
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"List the main features.",
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"Which LLM providers are supported?",
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"Summarize the document in 3 bullet points."
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
]
# Loop through each item so the same operation can be demonstrated for multiple inputs.
for q in questions:
# Display the result/status in the notebook so learners can see what happened.
print("=" * 70)
# Display the result/status in the notebook so learners can see what happened.
print("Question:", q)
# Display the result/status in the notebook so learners can see what happened.
print()
# Send the question to the query engine so it can retrieve relevant context and generate an
answer.
response = query_engine.query(q)
# Display the result/status in the notebook so learners can see what happened.
print(response)
# Display the result/status in the notebook so learners can see what happened.
print()

```

Output

```
=====
Question: What is Retrieval-Augmented Generation?
Retrieval-Augmented Generation is a technique that combines the ability to pull in relevant
information from external data sources with the language model's own text-generation
capabilities, so the output is enriched with up-to-date or specific knowledge drawn from those
sources.
=====
Question: List the main features.
- Retrieval-Augmented Generation (RAG)
- Vector database integration
- Document indexing
- AI agents
- Query engines
- Chat engines
=====
Question: Which LLM providers are supported?
OpenAI, Groq, Anthropic, Gemini, and numerous other LLM providers.
=====
Question: Summarize the document in 3 bullet points.
- LlamaIndex is an open-source framework that links large language models to external data
sources.
- It provides capabilities such as Retrieval-Augmented Generation, vector database
integration, document indexing, AI agents, query engines, and chat engines.
- The framework is compatible with major LLM providers including OpenAI, Groq, Anthropic,
Gemini, and others.
```

? Classroom Theory - Chat Engine

A query engine is generally question/answer oriented.

A chat engine is designed for conversational interaction.

Instead of repeatedly calling:

we can interact like:

```
```text
```

User: What is LlamaIndex?

Assistant:

User: What are its main features?

Assistant:

```
```
```

The chat engine provides a conversational interface around the indexed knowledge.

Interactive chat using LlamaIndex

Code

```
# Create a conversational engine so the application can handle interactive chat.
chat_engine = index.as_chat_engine()
# Keep the interactive chat loop running until the user enters the exit command.
while True:
    # Read the learner's question from the keyboard.
    question = input("\nAsk a question (type 'exit' to stop): ")
    # Check the stopping condition before deciding whether to leave the loop.
    if question.lower() == "exit":
        # Stop the current loop.
        break
    # Store the generated response so it can be printed or inspected.
    response = chat_engine.chat(question)
    # Display the result/status in the notebook so learners can see what happened.
    print("\nAssistant:")
    # Display the result/status in the notebook so learners can see what happened.
    print(response)
```

Output

```
Ask a question (type 'exit' to stop): exit
```

? Classroom Theory - Chunking

A document is usually too large to send completely to an LLM for every question.

So we split it into smaller pieces called chunks.

Example:

`chunk\_size`

Controls approximately how large each chunk should be.

`chunk\_overlap`

Keeps some content from the previous chunk in the next chunk.

Overlap helps avoid losing context at chunk boundaries.

Classroom question

Why not make one huge chunk?

Because retrieval becomes less precise and the amount of unnecessary context sent to the LLM increases.

Why not make chunks extremely tiny?

Because important context may be split apart.

So chunking is a practical trade-off between retrieval precision and context preservation.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# ## Chucking strateget comes from here.
# Existing comment/documentation line; it explains or labels the following code.
from llama_index.core.node_parser import SentenceSplitter
# Existing comment/documentation line; it explains or labels the following code.
from llama_index.core import Settings
# Existing comment/documentation line; it explains or labels the following code.
Settings.node_parser = SentenceSplitter(
# Existing comment/documentation line; it explains or labels the following code.
chunk_size=512,
# Existing comment/documentation line; it explains or labels the following code.
chunk_overlap=50
# Existing comment/documentation line; it explains or labels the following code.
)
```

? TOPIC 3 - Tree Index

? Classroom Theory - Tree Index

A Tree Index organizes information hierarchically.

Conceptually:

The tree structure can be useful for summary-style questions, where information from multiple pieces needs to be combined progressively.

Classroom takeaway

A vector index is naturally useful for semantic retrieval.

A tree index can be useful when we want to combine information hierarchically, especially for summarization.

Tree Index

Code

```
# Import TreeIndex from llama_index.core so we can use it directly.
from llama_index.core import TreeIndex
# Build a Tree Index, organizing document information hierarchically for summary-style
retrieval.
tree_index = TreeIndex.from_documents(documents, num_children=3)
# Display the result/status in the notebook so learners can see what happened.
print("Tree index built. The LLM summarized chunks bottom-up.")
```

Output

```
Tree index built. The LLM summarized chunks bottom-up.
```

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# Tree Index excels at SUMMARIZATION queries
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
tree_query_engine = tree_index.as_query_engine()
# Send the question to the query engine so it can retrieve relevant context and generate an
answer.
response = tree_query_engine.query(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"Give me a one-paragraph executive summary of NimbusCloud's policies and roadmap."
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
# Display the result/status in the notebook so learners can see what happened.
print("Q: Executive summary across all docs")
# Display the result/status in the notebook so learners can see what happened.
print("A:", response)
```

Output

Q: Executive summary across all docs

A: I'm sorry, but I don't have the information needed to provide an executive summary of

NimbusCloud's policies and roadmap.

Code

```
# Send the question to the query engine so it can retrieve relevant context and generate an answer.
response = tree_query_engine.query(
# This line performs the operation shown below; its role is explained by the surrounding classroom theory.
"Give me a one-line-summary about LlamaIndex"
# This line performs the operation shown below; its role is explained by the surrounding classroom theory.
)
# Display the result/status in the notebook so learners can see what happened.
print("Q: Executive summary across all docs")
# Display the result/status in the notebook so learners can see what happened.
print("A:", response)
```

Output

```
Q: Executive summary across all docs
A: LlamaIndex is an open-source framework that links large language models to external data,
```

enabling retrieval?augmented generation, vector databases, document indexing, AI agents, query engines, and chat engines.

? TOPIC 4 - Keyword Index

? Classroom Theory - Keyword Index

A keyword index focuses on important terms extracted from the document chunks.

This gives us another retrieval strategy:

Compare with vector search

Keyword search

- Strong when exact/important terms matter
- Easy to understand
- Can miss semantically similar wording

Vector search

- Strong for semantic similarity
- Can find related wording even when exact keywords differ

The notebook demonstrates that LlamaIndex supports multiple indexing approaches rather than forcing every problem into one retrieval strategy.

Code

```

# Import the os module so its functionality is available in this notebook.
import os
# Import the shutil module so its functionality is available in this notebook.
import shutil
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
DATA_DIR = "shopsphere_docs"
# Check the stopping condition before deciding whether to leave the loop.
if os.path.exists(DATA_DIR):
# Remove the existing directory and its contents so the demo starts cleanly.
shutil.rmtree(DATA_DIR)
# Create the directory if it does not already exist.
os.makedirs(DATA_DIR)
# Define the synthetic ShopSphere knowledge base as a mapping of filenames to document
contents.
docs = {
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"shipping_policy.txt": """
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
ShopSphere Shipping Policy.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Standard shipping takes 5 to 7 business days.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Express shipping delivers within 1 to 2 business days.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Orders above $75 qualify for free standard shipping.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
International shipping is available to more than 40 countries.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Customers receive a tracking number once the order has been dispatched.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
""",
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"return_policy.txt": """
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
ShopSphere Return and Exchange Policy.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Products may be returned within 30 days of delivery.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Items must be unused and returned in their original packaging.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Electronics can only be exchanged if they are defective.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Refunds are issued to the original payment method.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Gift cards are non-refundable.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.

```

Output

```
Created 5 synthetic documents in 'shopsphere_docs/'  
Files: ['shipping_policy.txt', 'membership_program.txt', 'warehouse_operations.txt',
```

```
'return_policy.txt', 'customer_support.txt']
```

Code

```
# Import SimpleDirectoryReader from llama_index.core so we can use it directly.
from llama_index.core import SimpleDirectoryReader
# Load the files from the data directory into LlamaIndex Document objects.
documents = SimpleDirectoryReader(DATA_DIR).load_data()
# Display the result/status in the notebook so learners can see what happened.
print(f"Loaded {len(documents)} documents.")
```

Error Output

```
WARNING:llama_index.core.readers.file.base:`llama-index-readers-file` package not found, some
file readers will not be available if not provided by the `file_extractor` parameter.
WARNING:llama_index.core.readers.file.base:`llama-index-readers-file` package not found, some
file readers will not be available if not provided by the `file_extractor` parameter.
WARNING:llama_index.core.readers.file.base:`llama-index-readers-file` package not found, some
```

```
file readers will not be available if not provided by the `file_extractor` parameter.  
WARNING:llama_index.core.readers.file.base:`llama-index-readers-file` package not found, some  
file readers will not be available if not provided by the `file_extractor` parameter.  
WARNING:llama_index.core.readers.file.base:`llama-index-readers-file` package not found, some  
file readers will not be available if not provided by the `file_extractor` parameter.
```

Output

```
Loaded 5 documents.
```

Code

```
# Existing comment/documentation line; it explains or labels the following code.  
# Build a Keyword Index over the same docs  
# Import SimpleKeywordTableIndex from llama_index.core so we can use it directly.  
from llama_index.core import SimpleKeywordTableIndex  
# Build a keyword index by extracting and organizing important terms from document chunks.  
keyword_index = SimpleKeywordTableIndex.from_documents(documents)  
# Display the result/status in the notebook so learners can see what happened.  
print("Keyword index built. Keywords extracted per chunk.")
```

Output

```
Keyword index built. Keywords extracted per chunk.
```

Code

```
# Display/inspect the keyword index object.  
keyword_index
```

Output

```
<llama_index.core.indices.keyword_table.simple_base.SimpleKeywordTableIndex at 0x788b6827e660>
```

Code

```
# Create a query engine on top of the keyword index.
keyword_query_engine = keyword_index.as_query_engine()
# Send the question to the query engine so it can retrieve relevant context and generate an
answer.
response = keyword_query_engine.query("What does the policy say reward points?")
# Display the result/status in the notebook so learners can see what happened.
print("Q: What does the policy say reward points?")
# Display the result/status in the notebook so learners can see what happened.
print("A:", response)
```

Output

```
Q: What does the policy say reward points?
A: Empty Response
```

? TOPIC 5 - Knowledge Graph

? Classroom Theory - Knowledge Graph Indexing

A knowledge graph represents information as entities and relationships.

Example:

This can be represented as triplets:

For example:

Why is this useful?

Some questions are naturally relational:

Who owns the service that Alices service depends on?

The answer requires following relationships:

This is different from simply looking for one matching sentence.

Common graph entities

- * People
- * Organizations
- * Products
- * News/events
- * Services
- * Systems

Classroom takeaway

Vector retrieval finds relevant text.

Knowledge graphs explicitly represent relationships between entities.

Knowledge Graph Indexing

Alice manages Payments Service.

Payments Service depends on Authentication Service.

Authentication Service is owned by Bob.

Code

```
# Existing comment/documentation line; it explains or labels the following code.  
# Who owns the service that Alice's service depends on?
```

(subject, relationship, object)

Code

```
# Existing comment/documentation line; it explains or labels the following code.  
# (Alice, manages, PaymentService) # Chunk 1  
# Existing comment/documentation line; it explains or labels the following code.  
# (PaymentService, depends_on, AuthService) # chunk 2  
# Existing comment/documentation line; it explains or labels the following code.  
# (AuthService, owned_by, Bob) # chunk 3
```

Code

```
# Existing comment/documentation line; it explains or labels the following code.  
# Examples  
# Existing comment/documentation line; it explains or labels the following code.  
# 1. People  
# Existing comment/documentation line; it explains or labels the following code.  
# 2. Organization  
# Existing comment/documentation line; it explains or labels the following code.  
# 3. Products  
# Existing comment/documentation line; it explains or labels the following code.  
# 4. news
```

Code

```
# Install the required Python package in the current notebook environment.  
!pip install wikipedia
```

Output

```
Collecting wikipedia
Downloading wikipedia-1.4.0.tar.gz (27 kB)
Preparing metadata (setup.py) ... [?25l [?25hdone
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/dist-packages (from wikipedia) (4.13.5)
Requirement already satisfied: requests<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from wikipedia) (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.0.0->wikipedia) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.0.0->wikipedia) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.0.0->wikipedia) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.0.0->wikipedia) (2026.7.22)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->wikipedia) (2.9.2)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->wikipedia) (4.16.0)
```

? Classroom Theory - Building a Knowledge Graph from Text

The Amazon example provides a larger natural-language document.

The goal is to let LlamaIndex extract relationships from that text.

Conceptually:

For example, the text may contain relationships conceptually similar to:

The exact extracted triplets depend on the source text and the LLMs extraction.

Code

```

# Import Document from llama_index.core so we can use it directly.
from llama_index.core import Document
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
amazon_text = """
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Amazon.com, Inc. is an American multinational technology company headquartered
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
in Seattle, Washington. The company was founded by Jeff Bezos in 1994 as an
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
online bookstore before expanding into e-commerce, cloud computing, digital
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
streaming, and artificial intelligence.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Amazon Web Services (AWS) is the company's cloud computing division and is one
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
of the world's leading cloud platforms. Amazon operates fulfillment centers
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
across multiple countries and provides services such as Amazon Prime, Prime
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Video, Kindle, Alexa, and Audible.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
The company acquired Whole Foods Market in 2017, expanding its presence in the
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
grocery industry. Amazon also owns Twitch, an online live-streaming platform,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
and MGM Studios.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Andy Jassy succeeded Jeff Bezos as the Chief Executive Officer in 2021 after
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
previously leading AWS. Amazon continues to invest heavily in robotics,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
artificial intelligence, logistics, and satellite internet through Project
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Kuiper.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
Amazon's major business segments include online retail, cloud computing,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
advertising, subscription services, and third-party marketplace operations.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
The company competes with Microsoft, Google, Walmart, and Alibaba in various
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
markets worldwide.
# This line performs the operation shown below; its role is explained by the surrounding

```

Output

Created 1 document.

Code

```
# This line performs the operation shown below; its role is explained by the surrounding  
classroom theory.  
amazon_docs
```

Output

```
[Document(id_='b4999484-ac1e-4087-89df-d0122b1b7c44', embedding=None, metadata={},
```

```
excluded_embed_metadata_keys=[], excluded_llm_metadata_keys=[], relationships={},
```

```
metadata_template='{key}: {value}', metadata_separator='\n',
```

```
text_resource=MediaResource(embeddings=None, data=None, text="\n# This line performs the
```

operation shown below; its role is explained by the surrounding classroom theory.\nAmazon.com,

Inc. is an American multinational technology company headquartered\n# This line performs the

operation shown below; its role is explained by the surrounding classroom theory.\nin Seattle,

Washington. The company was founded by Jeff Bezos in 1994 as an\n# This line performs the

operation shown below; its role is explained by the surrounding classroom theory.\nonline

bookstore before expanding into e-commerce, cloud computing, digital\n# This line performs the

operation shown below; its role is explained by the surrounding classroom theory.\nstreaming,

and artificial intelligence.\n\n# This line performs the operation shown below; its role is

explained by the surrounding classroom theory.\nAmazon Web Services (AWS) is the company's

cloud computing division and is one\n# This line performs the operation shown below; its role

is explained by the surrounding classroom theory.\nof the world's leading cloud platforms.

Amazon operates fulfillment centers\n# This line performs the operation shown below; its role

is explained by the surrounding classroom theory.\nacross multiple countries and provides

services such as Amazon Prime, Prime\n# This line performs the operation shown below; its role

is explained by the surrounding classroom theory.\nVideo, Kindle, Alexa, and Audible.\n\n#

This line performs the operation shown below; its role is explained by the surrounding

classroom theory.\n\nThe company acquired Whole Foods Market in 2017, expanding its presence in

the\n# This line performs the operation shown below; its role is explained by the surrounding

classroom theory.\ngrocery industry. Amazon also owns Twitch, an online live-streaming

platform,\n# This line performs the operation shown below; its role is explained by the

surrounding classroom theory.\nand MGM Studios.\n\n# This line performs the operation shown

below; its role is explained by the surrounding classroom theory.\nAndy Jassy succeeded Jeff

Bezos as the Chief Executive Officer in 2021 after\n# This line performs the operation shown

below; its role is explained by the surrounding classroom theory.\npreviously leading AWS.

Amazon continues to invest heavily in robotics,\n# This line performs the operation shown

below; its role is explained by the surrounding classroom theory.\nartificial intelligence,

logistics, and satellite internet through Project\n# This line performs the operation shown

below; its role is explained by the surrounding classroom theory.\nKuiper.\n\n# This line

performs the operation shown below; its role is explained by the surrounding classroom

theory.\nAmazon's major business segments include online retail, cloud computing,\n# This line

performs the operation shown below; its role is explained by the surrounding classroom

theory.\nadvertising, subscription services, and third-party marketplace operations.\n# This

line performs the operation shown below; its role is explained by the surrounding classroom

theory.\n\nThe company competes with Microsoft, Google, Walmart, and Alibaba in various\n\n# This

line performs the operation shown below; its role is explained by the surrounding classroom

theory.\nmarkets worldwide.\n# This line performs the operation shown below; its role is

explained by the surrounding classroom theory.\n", path=None, url=None, mimetype=None),

image_resource=None, audio_resource=None, video_resource=None,

```
text_template='{metadata_str}\n\n{content}']
```

? Classroom Theory - KnowledgeGraphIndex and GraphStore

``SimpleGraphStore``

Stores the extracted graph relationships.

``StorageContext``

Connects the graph store to the LlamaIndex storage configuration.

``KnowledgeGraphIndex``

Processes the documents and extracts graph-oriented knowledge.

``max_triplets_per_chunk``

Controls how many triplets are extracted from each chunk.

``include_embeddings=True``

Also enables embeddings for the indexed knowledge, allowing semantic retrieval to complement the graph structure.

Why does this take longer?

Unlike a simple vector index, the LLM may need to inspect text and infer entities and relationships before the graph can be built.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# Build the Knowledge Graph Index
# Import KnowledgeGraphIndex from llama_index.core so we can use it directly.
from llama_index.core import KnowledgeGraphIndex
# Import SimpleGraphStore from llama_index.core.graph_stores so we can use it directly.
from llama_index.core.graph_stores import SimpleGraphStore
# Import StorageContext from llama_index.core so we can use it directly.
from llama_index.core import StorageContext
# Create an in-memory graph store for keeping the extracted relationships.
graph_store = SimpleGraphStore()
# Create the LlamaIndex storage context and attach the graph store to it.
storage_context = StorageContext.from_defaults(graph_store=graph_store)
# Build a knowledge graph by extracting entities and relationships from the documents.
kg_index = KnowledgeGraphIndex.from_documents(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
amazon_docs,
# Limit how many knowledge-graph triplets are extracted from each document chunk.
max_triplets_per_chunk=5,
# Tell the knowledge-graph index where the extracted graph data should be stored.
storage_context=storage_context,
# Also create embeddings so graph-related retrieval can use semantic similarity.
include_embeddings=True,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
# Display the result/status in the notebook so learners can see what happened.
print("Knowledge Graph Index built.")
# Display the result/status in the notebook so learners can see what happened.
print("This took longer because the LLM had to extract entities and relations from each
chunk.")
```

Output

Knowledge Graph Index built.
This took longer because the LLM had to extract entities and relations from each chunk.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# Peek at the extracted triplets
# Retrieve the graph's relationship map so we can inspect extracted subject-relation-object triplets.
rel_map = graph_store.get_rel_map()
# Display the result/status in the notebook so learners can see what happened.
print("Sample triplets extracted from amazon article:")
# Keep track of how many relationships have been displayed.
count = 0
# Loop through each item so the same operation can be demonstrated for multiple inputs.
for entity, relations in rel_map.items():
    # Loop through each item so the same operation can be demonstrated for multiple inputs.
    for rel in relations[:2]:
        # Existing comment/documentation line; it explains or labels the following code.
        # rel is typically a list/tuple like [subject, relation, object]
        # Try the normal triplet-printing logic; the structure may vary slightly by LlamaIndex version.
        try:
            # Display the result/status in the notebook so learners can see what happened.
            print(f" {entity} --[{rel[1]}]--> {rel[2]}")
        except Exception:
            # Display the result/status in the notebook so learners can see what happened.
            print(f" {entity} -> {rel}")
        # Keep track of how many relationships have been displayed.
        count += 1
    # Check the stopping condition before deciding whether to leave the loop.
    if count >= 10:
        # Stop the current loop.
        break
# Check the stopping condition before deciding whether to leave the loop.
if count >= 10:
    # Stop the current loop.
    break
```

Output

```
Sample triplets extracted from amazon article:  
Amazon --[Acquired]--> Whole?foods?market  
Andy?jassy --[Succeeded]--> Jeff?bezos
```

? TOPIC 6 - Sub-Question Query Engine

? Classroom Theory - Sub-Question Query Engine

Complex questions often contain multiple smaller questions.

For example:

Compare the founders of Tesla and OpenAI.

This can be decomposed into:

This is the idea behind Sub-Question Querying.

It is especially useful when information is distributed across different sources or tools.

Sub Question Query Engine

? Classroom Theory - Multiple Knowledge Sources

Here we create separate corpora for multiple companies.

Conceptually:

Each source has its own searchable index.

The system can then expose these query engines as tools.

This is an important step toward agentic behavior because the LLM can decide which knowledge source should answer which part of a question.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# Fetch 2 more Wikipedia articles for cross-document comparison
# Define the external company articles that will be used in the multi-source example.
companies_to_fetch = ["Netflix", "OpenAI"]
# Create a dictionary that will hold a separate document collection for each company.
company_docs = {}
# Loop through each item so the same operation can be demonstrated for multiple inputs.
for company in companies_to_fetch:
    # Try the normal triplet-printing logic; the structure may vary slightly by LlamaIndex
    version.
    try:
        # Fetch the selected Wikipedia page so its content can be used as source data.
        page = wikipedia.page(company, auto_suggest=False)
        # Wrap raw text as a LlamaIndex Document object.
        company_docs[company] = [Document(text=page.content[:8000])]
        # Display the result/status in the notebook so learners can see what happened.
        print(f"Fetched {company} ({len(page.content[:8000])} chars)")
        # If the relationship structure is different, use a simpler fallback representation.
        except Exception as e:
            # Display the result/status in the notebook so learners can see what happened.
            print(f"Fallback for {company}: {e}")
            # Wrap raw text as a LlamaIndex Document object.
            company_docs[company] = [Document(text=f"{company} is a major technology company.")]
        # Existing comment/documentation line; it explains or labels the following code.
        # Also reuse Tesla docs from earlier
        # Create a dictionary that will hold a separate document collection for each company.
        company_docs["Tesla"] = tesla_docs
        # Display the result/status in the notebook so learners can see what happened.
        print("\nAll three company corpora ready.")
```

Output

```

    Fetched Netflix (8000 chars)
    Fetched OpenAI (8000 chars)
    All three company corpora ready.
```

Code

```

# Count the number of items so learners can verify the size of the collection.
len(company_docs)
```

Output

```

3
```

? Classroom Theory - QueryEngineTool

A wraps a query engine and makes it available as a tool.

Think of a tool as:

The planner does not need to know how the underlying vector index works.

It only needs to know:

```

    This tool can answer Tesla-related questions.
```

Why metadata matters

The identifies the tool.

The tells the planner what the tool is good at.

Good tool descriptions improve tool selection.

Code

```

# Existing comment/documentation line; it explains or labels the following code.
# Build a separate vector index per company, then expose each as a tool
# Import QueryEngineTool, ToolMetadata from llama_index.core.tools so we can use it directly.
from llama_index.core.tools import QueryEngineTool, ToolMetadata
# Store the query engine for the current company so it can be exposed as a tool.
company_engines = {}
# Loop through each item so the same operation can be demonstrated for multiple inputs.
for company, docs in company_docs.items():
# Create a vector-based index from the documents.
idx = VectorStoreIndex.from_documents(docs)
# Store the query engine for the current company so it can be exposed as a tool.
company_engines[company] = idx.as_query_engine(similarity_top_k=2)
# Create the list of query-engine tools available to the sub-question planner.
query_engine_tools = [
# Wrap a query engine as a tool that another LlamaIndex component can call.
QueryEngineTool(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
query_engine=company_engines["Tesla"],
# Describe the tool so the planner can understand what information it provides.
metadata=ToolMetadata(
# Give the tool a stable logical name that the planner can refer to.
name="tesla_docs",
# Explain what information this tool contains so the planner can select it correctly.
description="Information about Tesla, Inc.: founding, products, headquarters,
executives."
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
),
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
),
# Wrap a query engine as a tool that another LlamaIndex component can call.
QueryEngineTool(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
query_engine=company_engines["Netflix"],
# Describe the tool so the planner can understand what information it provides.
metadata=ToolMetadata(
# Give the tool a stable logical name that the planner can refer to.
name="netflix_docs",
# Explain what information this tool contains so the planner can select it correctly.
description="Information about Netflix: founding, business model, content,
executives."
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
),
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
),
# Wrap a query engine as a tool that another LlamaIndex component can call.
QueryEngineTool(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
query_engine=company_engines["OpenAI"],
# Describe the tool so the planner can understand what information it provides.
metadata=ToolMetadata(
# Give the tool a stable logical name that the planner can refer to.
name="openai_docs",
# Explain what information this tool contains so the planner can select it correctly.
description="Information about OpenAI: founding, mission, products like ChatGPT
and GPT-4, leadership."
# This line performs the operation shown below; its role is explained by the surrounding

```

Output

```
Built 3 query-engine tools.
```

? Classroom Theory - Sub-Question Planning

The sub-question engine combines:

- Multiple query-engine tools
- A question generator
- The LLM
- A final response-combination step

The conceptual workflow is:

This is one of the bridges between RAG and agentic/tool-based AI.

Code

```

# Existing comment/documentation line; it explains or labels the following code.
# -----
# Existing comment/documentation line; it explains or labels the following code.
# Sub-Question Query Engine
# Existing comment/documentation line; it explains or labels the following code.
# -----
# Create a question generator that uses the configured LLM to decompose complex questions.
question_gen = LLMQuestionGenerator.from_defaults(
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
llm=Settings.llm
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
# Create an engine that breaks a complex question into smaller sub-questions and routes them
to tools.
sub_question_engine = SubQuestionQueryEngine.from_defaults(
# Create the list of query-engine tools available to the sub-question planner.
query_engine_tools=query_engine_tools,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
question_gen=question_gen,
# Disable asynchronous execution for a simpler classroom demonstration.
use_async=False,
# Enable detailed logging so learners can see the sub-question process.
verbose=True,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
# Display the result/status in the notebook so learners can see what happened.
print("Sub-Question Query Engine ready.")

```

Output

Sub-Question Query Engine ready.

Code

```
# Existing comment/documentation line; it explains or labels the following code.
# -----
# Existing comment/documentation line; it explains or labels the following code.
# Example Queries
# Existing comment/documentation line; it explains or labels the following code.
# -----
# Define the complex questions that will be tested with sub-question decomposition.
queries = [
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"Compare the founders of Tesla and OpenAI."
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
]
# Loop through each item so the same operation can be demonstrated for multiple inputs.
for q in queries:
# Display the result/status in the notebook so learners can see what happened.
print("=" * 80)
# Display the result/status in the notebook so learners can see what happened.
print("Question:", q)
# Send the question to the query engine so it can retrieve relevant context and generate an
answer.
response = sub_question_engine.query(q)
# Display the result/status in the notebook so learners can see what happened.
print(response)
```

Output

```
=====
Question: Compare the founders of Tesla and OpenAI.
```

Code

```
# Read/set the API-key environment variable so the LLM can authenticate without hard-coding
the secret.
os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")
```

? Classroom Theory - LLM-Based Structured Planning

The final section demonstrates a different idea: asking the LLM to produce a structured plan.

Instead of directly asking:

Give me the answer.

we ask:

Which knowledge sources should I use, and what question should I ask each source?

The expected structure is JSON:

This is useful because software can parse structured JSON and execute the requested tools programmatically.

Classroom takeaway

This is a simple pattern for building an agent:

The notebooks final planning example assumes , , and have already been created elsewhere.

Code

```

# Import Groq from llama_index.llms.groq so we can use it directly.
from llama_index.llms.groq import Groq
# Import Settings from llama_index.core so we can use it directly.
from llama_index.core import Settings
# Configure the global LlamaIndex setting used by subsequent indexing/query operations.
Settings.llm = Groq(
# Specify the LLM model name used for generation.
model="llama-3.3-70b-versatile",
# Pass the API key obtained from the environment rather than embedding a secret in source
code.
api_key=os.environ["GROQ_API_KEY"],
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
)
# Existing comment/documentation line; it explains or labels the following code.
# -----
# Existing comment/documentation line; it explains or labels the following code.
# Build your three indexes as usual
# Existing comment/documentation line; it explains or labels the following code.
# -----
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
python_engine = python_index.as_query_engine()
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
pandas_engine = pandas_index.as_query_engine()
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
numpy_engine = numpy_index.as_query_engine()
# Map logical tool names to their corresponding query engines.
engines = {
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"python_docs": python_engine,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"pandas_docs": pandas_engine,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"numpy_docs": numpy_engine,
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
}
# Describe each available knowledge source so the LLM planner can choose the right one.
tool_descriptions = """
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
python_docs : Questions about Python programming language.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
pandas_docs : Questions about Pandas DataFrames.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
numpy_docs : Questions about NumPy arrays and numerical computing.
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"""
# Store the user's complex question that needs planning.
user_question = (
# This line performs the operation shown below; its role is explained by the surrounding
classroom theory.
"Compare Python, Pandas and NumPy. "
# This line performs the operation shown below; its role is explained by the surrounding

```

Output

```
```json
{
 "items": [
 {
 "tool": "python_docs",
 "question": "How does Python relate to data analysis libraries?"
 },
 {
 "tool": "pandas_docs",
 "question": "What is the role of Pandas in data manipulation and analysis?"
 },
 {
 "tool": "numpy_docs",
 "question": "How does NumPy contribute to numerical computing in Python?"
 },
 {
 "tool": "python_docs",
 "question": "How do Pandas and NumPy integrate with Python for data science tasks?"
 }
]
}
```
```

? Final Classroom Recap - LlamaIndex + RAG

What students should remember

1. LlamaIndex

A framework that helps connect LLMs with external/private data.

2. RAG

Retrieve relevant information first, then generate an answer using that retrieved context.

3. Embeddings

Convert text into vectors so semantic similarity can be measured.

4. Vector Index

Stores/searches document representations for semantic retrieval.

5. Query Engine

Provides a question-answering interface over an index.

6. Chat Engine

Provides a conversational interface over the indexed data.

7. Chunking

Breaks large documents into smaller pieces so retrieval is more precise and context can be managed.

8. Tree Index

Organizes information hierarchically and is useful for summary-oriented workflows.

9. Keyword Index

Uses important terms/keywords for retrieval.

10. Knowledge Graph

Represents entities and relationships as structured connections/triplets.

11. Sub-Question Querying

Breaks a complex question into smaller questions and combines their answers.

12. Query Engine Tools

Expose individual knowledge sources as tools that a planner can select.

13. Structured Planning

An LLM can produce a machine-readable plan, such as JSON, describing which tools to call and what questions to ask.

? One complete mental model

Suggested trainer closing statement

We started with a simple RAG application, where LlamaIndex loads documents, creates embeddings, builds an index, retrieves relevant chunks and sends them to an LLM. Then we explored alternative indexing strategies such as Tree, Keyword and Knowledge Graph indexes. Finally, we moved toward agentic behavior by using multiple query engines as tools, decomposing complex questions into sub-questions, and generating structured tool-selection plans.